

LAB SESSION 10

Lab Name: Maintenance, Ethics & Final Review

Project: MediScript-E — Digital Healthcare Platform

Objectives

- Understand post-deployment software challenges for MediScript-E
- Address ethical responsibilities in healthcare software development
- Present final project review with complete SDLC summary

Theory

Most software cost occurs during maintenance. After deployment, software must be updated, patched, and improved continuously. Engineers must act ethically, protecting users and data — especially in healthcare systems where patient safety and privacy are paramount.

Task 1: Maintenance Plan

1.1 Types of Maintenance

Type	Description	Examples in MediScript-E
Corrective	Fix bugs and errors discovered after deployment	Fix SSL connection issues, fix OTP expiry bugs
Adaptive	Adapt to changes in environment or dependencies	Prisma 7 migration, Next.js version upgrades
Perfective	Improve performance and add new features	Add 2FA, optimize database queries, add search
Preventive	Prevent future failures through refactoring	Connection pooling, error handling improvements

1.2 Scheduled Maintenance Activities

Activity	Frequency	Responsible	Description
Dependency updates	Monthly	Developer	Update npm packages, check for security patches
Security audit	Monthly	Developer	Run <code>npm audit</code> , fix vulnerabilities

Activity	Frequency	Responsible	Description
Database backup verification	Weekly	Supabase (Auto)	Verify Supabase automatic backups are working
Production log review	Weekly	Developer	Review Vercel logs for errors and anomalies
Environment variable rotation	Quarterly	Developer	Rotate NEXTAUTH_SECRET, CRON_API_KEY
OAuth credential review	Quarterly	Developer	Verify Google and GitHub OAuth apps are active
Performance monitoring	Monthly	Developer	Check Vercel analytics for slow API routes
Dead code cleanup	Quarterly	Developer	Remove unused components and API routes

1.3 Version Control & Release Strategy

Strategy	Implementation
Branch	Single main branch for production
Deployment	Vercel auto-deploys on every push to main
Rollback	Vercel dashboard allows instant rollback to previous deployment
Hotfix	Direct commit to main for critical production fixes
Feature development	Local development → test → commit → push → auto-deploy

1.4 Known Technical Debt & Future Improvements

Item	Priority	Description
Rate limiting on API routes	High	Prevent brute force and DDoS attacks on login and OTP endpoints
Automated test suite (Jest)	High	Replace manual testing with automated unit and integration tests
Email queue system	Medium	Use a proper email queue (e.g., Bull) instead of direct SMTP calls
Refresh token rotation	Medium	Implement refresh token rotation for enhanced session security
Doctor availability calendar	Medium	Add time slot management for doctors
Notification system	Medium	Real-time notifications using WebSockets or Server-Sent Events
Audit logging	Medium	Log all admin actions for accountability

Item	Priority	Description
TOTP-based 2FA	Low	Add Google Authenticator support as alternative to email OTP
Multi-language support	Low	Add Bengali language support for local users
Mobile app	Low	React Native app for patients and doctors

1.5 Monitoring & Alerting

Tool	Purpose	Current Status
Vercel Analytics	Page load performance, visitor tracking	Active
Vercel Logs	Runtime errors, API response times	Active
Supabase Dashboard	Database query performance, storage usage	Active
GitHub	Code changes, commit history	Active
Manual testing	Feature verification after each deployment	Active

Task 2: Ethics Analysis

2.1 Ethical Responsibilities in Healthcare Software

As developers of MediScript-E, we have ethical responsibilities to:

- **Protect patient privacy:** Medical data is highly sensitive and must be secured
- **Ensure system reliability:** Downtime or bugs can affect patient care
- **Prevent unauthorized access:** Only authorized users should access medical records
- **Be transparent:** Users should know how their data is used
- **Avoid harm:** System errors must not lead to incorrect medical decisions

2.2 Ethical Issues Identified

Issue ID	Ethical Issue	Category	Impact	Mitigation
E-01	Patient medical data stored in cloud (Supabase)	Privacy	High	SSL encryption, access control, Supabase security policies
E-02	Email addresses used for OTP and reminders	Privacy	Medium	Emails used only for system notifications, never shared
E-03	Admin can view and delete any user account	Power misuse	High	Admin actions logged, self-deletion prevented
E-04	OAuth login auto-creates patient accounts	Consent	Medium	Users explicitly click OAuth button — implied consent

Issue ID	Ethical Issue	Category	Impact	Mitigation
E-05	Prescription data accessible to both doctor and patient	Data sharing	Medium	RBAC ensures only relevant parties access prescriptions
E-06	Medicine reminder emails sent automatically	Autonomy	Low	Users set reminders themselves, can delete anytime
E-07	System downtime could delay medical access	Reliability	High	99.9% uptime target, Vercel auto-scaling
E-08	Fake doctor registration with invalid license	Fraud	Critical	License number uniqueness enforced, manual verification recommended
E-09	Patient blood group stored without explicit consent notice	Informed consent	Medium	Blood group is optional profile data, user selects during registration
E-10	No data deletion/export feature for users (GDPR)	User rights	Medium	Future improvement: add account deletion and data export

2.3 Privacy & Data Protection Measures

Measure	Implementation
Data minimization	Only necessary data collected (no unnecessary PII)
Password protection	bcryptjs hashing — passwords never stored in plaintext
Secure transmission	HTTPS enforced on all routes via Vercel
Database encryption	SSL/TLS connection to Supabase PostgreSQL
Access control	RBAC — users only access their own data
Token expiry	Verification tokens (24h), OTP (10min), sessions (30 days)
Environment secrets	All credentials in <code>.env</code> — never in source code
File security	Medical documents stored in Supabase with access control

2.4 ACM Code of Ethics — Applied to MediScript-E

ACM Principle	Application in MediScript-E
Contribute to society and human well-being	MediScript-E improves healthcare access for patients and doctors
Avoid harm	System tested thoroughly to prevent incorrect medical data

ACM Principle	Application in MediScript-E
Be honest and trustworthy	No fake features — all documented features are implemented
Respect privacy	Patient medical data protected with encryption and RBAC
Honor confidentiality	Medical records accessible only to authorized users
Give comprehensive and thorough evaluations	All modules tested with 56 test cases before deployment

Task 3: Final Project Report

3.1 Project Summary

Field	Details
Project Name	MediScript-E
Type	Digital Healthcare Platform
Live URL	https://mediscript-e.vercel.app
Repository	https://github.com/tusharsno/mediscript-e
Developer	Tushar
University	USTC, Chittagong
Tech Stack	Next.js 16, React 19, TypeScript, Prisma 7, PostgreSQL, Supabase, NextAuth, Vercel

3.2 SDLC Summary

Phase	Lab Session	Deliverable	Status
Requirements Engineering	Lab 3	FR List, NFR List, Priority Matrix	✓ Complete
SRS Documentation	Lab 4	IEEE-standard SRS Document	✓ Complete
Use Case Modeling	Lab 5	Use Case Diagram, 12 Use Case Specifications	✓ Complete
System Design	Lab 6	Architecture Diagram, DB Schema, UML Diagrams, API Design	✓ Complete
Security Engineering	Lab 7	Failure Scenarios, Threat Analysis, Security Controls, RBAC Matrix	✓ Complete
Testing	Lab 8	Test Plan, 56 Test Cases, RTM	✓ Complete

Phase	Lab Session	Deliverable	Status
Implementation	Lab 9	Source Code, Git Repository, README	✓ Complete
Maintenance & Ethics	Lab 10	Maintenance Plan, Ethics Analysis, Final Report	✓ Complete

3.3 Features Implemented

Feature	Status	Notes
User Registration (Patient/Doctor)	✓	With blood group and license number
Email Verification	✓	24-hour token expiry
Credential Login	✓	bcryptjs password hashing
Google OAuth Login	✓	Auto-creates patient account
GitHub OAuth Login	✓	Auto-creates patient account
Two-Factor Authentication (2FA)	✓	Email OTP, 10-minute expiry
Appointment Booking	✓	Patient books with doctor, date, time
Appointment Management	✓	Doctor confirms/cancels/completes
Digital Prescriptions	✓	Doctor issues, patient views
PDF Download	✓	html2canvas + jsPDF
Medicine Reminders	✓	With automated email alerts via cron
Medical Vault	✓	Supabase Storage file upload
Admin Dashboard	✓	Real-time stats, user management
Settings (Profile/Password/2FA)	✓	Full account management
Contact Form	✓	Public form with admin view
Responsive Design	✓	Mobile, tablet, desktop
Production Deployment	✓	Vercel with custom domain

3.4 Project Statistics

Metric	Value
Total API Endpoints	30+
Database Models	8
React Components	20+

Metric	Value
Functional Requirements	36
Non-Functional Requirements	21
Use Cases	12
Test Cases	56
Security Threats Identified	15
Failure Scenarios Identified	15
Git Commits	15+
Lines of Code (approx.)	5000+

3.5 Challenges & Solutions

Challenge	Solution
Prisma 7 breaking changes on Vercel	Moved connection URLs to <code>prisma.config.ts</code> , used <code>@prisma/adapters-pg</code>
SSL/TLS certificate error on Supabase	Added <code>sslmode=no-verify</code> to DATABASE_URL
Vercel deploying old commit	Used <code>vercel --prod</code> CLI to bypass GitHub webhook issue
2FA bypass security vulnerability	Added <code>twoFactorCode !== null</code> check before session creation
<code>useSearchParams()</code> Suspense error	Wrapped component in <code><Suspense></code> boundary
Prisma engines download timeout	Used Supabase SQL Editor for direct migration

3.6 Lessons Learned

- **Requirements first:** Clear requirements in Lab 3 & 4 prevented scope creep during implementation
- **Security by design:** Implementing RBAC and 2FA from the start is easier than adding later
- **Version control is critical:** Git allowed safe rollback when Vercel deployment broke
- **Production ≠ Development:** SSL, environment variables, and serverless constraints behave differently in production
- **Documentation matters:** SRS and use cases served as reference during implementation and debugging
- **Test early:** Security test cases (TC-52 to TC-56) revealed RBAC gaps that were fixed before final deployment

Key Findings / Learning Outcomes

- Understood that **maintenance is the longest and most expensive phase** of the software lifecycle

- Learned that healthcare software carries **higher ethical responsibility** than general software due to sensitive medical data
- Recognized that **GDPR compliance** (data deletion, export) is a future requirement for MediScript-E
- Understood that **fake doctor registration** is a critical ethical risk that requires manual verification beyond technical controls
- Completed a full **SDLC cycle** from requirements engineering to deployment and maintenance planning
- Recognized that **ethical software engineering** means building systems that are not only functional but also safe, private, and trustworthy